

SOUGATA SAHA
IT-A2-037
OOS ASSIGNMENT 2

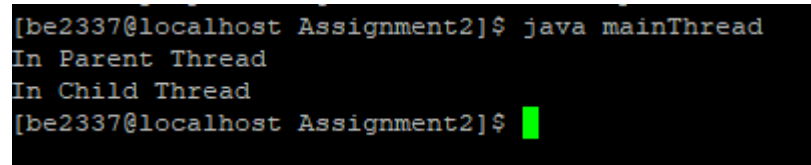
ASSIGNMENT 2

Q1. Write a program to create two threads. Print “In main thread” in main thread and “In child thread” in child thread.

Ans:

```
class childThread extends Thread {  
    public void run() {  
        System.out.println("In Child Thread");  
    }  
}  
  
class mainThread {  
    public static void main(String[] args) {  
        childThread ob = new childThread();  
        ob.start();  
        System.out.println("In Parent Thread");  
    }  
}
```

Output:



```
[be2337@localhost Assignment2]$ java mainThread  
In Parent Thread  
In Child Thread  
[be2337@localhost Assignment2]$
```

Q2. Create two threads and call them EvenThread and OddThread. EvenThread will print number as 2 4 6 8 10... and Odd Thread will print number as 1 3 5.... Now, synchronize these two threads to get the output as: 1 2 3 4 5 6 7 8.

Ans:

```
class Print {  
    synchronized void printEven() {  
        try {  
            wait();  
        }  
        catch(Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```

        for(int i=2; i<=8; i=i+2) {
            System.out.print(i + " ");
            notify();
            try
            {
                if(i<8) wait();
            }
            catch(Exception e)
            {
                e.printStackTrace();
            }
        }
    }

    synchronized void printOdd() {
        for(int i=1; i<=7; i=i+2) {
            System.out.print(i + " ");
            notify();
            try
            {
                if(i<7) wait();
            }
            catch(Exception e)
            {
                e.printStackTrace();
            }
        }
    }
}

class EvenThread extends Thread {
    Print p;
    EvenThread(Print p) {
        this.p = p;
    }
}

```

```

    }

    public void run() {
        p.printEven();
    }
}

class OddThread extends Thread {
    Print p;
    OddThread(Print p) {
        this.p = p;
    }
    public void run() {
        p.printOdd();
    }
}

class mainThread {
    public static void main(String[] args) {
        Print ob = new Print();
        EvenThread ob1 = new EvenThread(ob);
        OddThread ob2 = new OddThread(ob);
        ob1.start();
        ob2.start();
    }
}

```

Output:

```

[be2337@localhost Assignment2]$ java mainThread
1 2 3 4 5 6 7 8 [be2337@localhost Assignment2]$

```

3. Consider the following series $x = 1 + 1/1! + 1/2! + 1/3! + \dots + 1/10!$. Create two threads t1 & t2. t1 will generate the denominators and t2 will form the term and add them up. Finally print the result.

Ans:

```

class Series {
    double denominator = 1.0;
    double result = 1.0;
}

```

```

boolean isDenominatorReady = false;

double factorial(int n) {
    long ans = 1;
    for (int i = 1; i <= n; i++) {
        ans *= i;
    }
    return ans;
}

synchronized void generateDenominator() {
    for (int i = 1; i <= 10; i++) {
        denominator = factorial(i);
        isDenominatorReady = true;
        notify();
        try {
            if (i < 10) wait();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

synchronized void addResult() {
    for (int i = 1; i <= 10; i++) {
        try {
            while (!isDenominatorReady) wait();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        result += 1.0 / denominator;
        isDenominatorReady = false;
        notify();
    }
    System.out.println("Result: " + result);
}

```

```

    }
}
class DenominatorThread extends Thread {
    Series s;
    DenominatorThread(Series s) {
        this.s = s;
    }
    public void run() {
        s.generateDenominator();
    }
}
class ResultThread extends Thread {
    Series s;
    ResultThread(Series s) {
        this.s = s;
    }
    public void run() {
        s.addResult();
    }
}
class Demo {
    public static void main(String[] args) {
        Series series = new Series();
        DenominatorThread thread1 = new DenominatorThread(series);
        ResultThread thread2 = new ResultThread(series);
        thread1.start();
        thread2.start();
    }
}

```

Output:

```
[be2337@localhost Assignment2]$ javac q3.java
[be2337@localhost Assignment2]$ vi q3.java
[be2337@localhost Assignment2]$ java Demo
Result: 2.7182818011463845
[be2337@localhost Assignment2]$
```

Q4. Consider a file that contains a number of integers. Create two threads. Call them 'producer' and 'consumer' thread. Producer thread will be reading the integers from the file continuously while consumer thread will add them up. Use proper synchronization mechanism if needed.

Ans:

```
import java.io.*;

class FileProcessor {
    int sum = 0;
    int num;
    boolean isDone = false;
    boolean isProduced = false;
    boolean isNumber(char i) {
        return i >= '0' && i <= '9';
    }
    synchronized void readFile() {
        try (FileReader fr = new FileReader("numbers.txt")) {
            int i;
            while ((i = fr.read()) != -1) {
                if (isNumber((char) i)) {
                    num = (char) i - '0';
                    isProduced = true;
                    notify();
                    while (isProduced) {
                        wait();
                    }
                }
            }
            isDone = true;
            notify();
        } catch (IOException | InterruptedException e) {
```

```

        e.printStackTrace();
    }
}

synchronized void addNum() {
    try {
        while (!isDone) {
            while (!isProduced && !isDone) {
                wait();
            }
            if (isProduced) {
                sum += num;
                isProduced = false;
                notify();
            }
        }
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}

int getSum() {
    return sum;
}

}

class Producer extends Thread {
    private FileProcessor f;
    Producer(FileProcessor f) {
        this.f = f;
    }
    public void run() {
        f.readFile();
    }
}

```

```

class Consumer extends Thread {
    private FileProcessor f;
    Consumer(FileProcessor f) {
        this.f = f;
    }
    public void run() {
        f.addNum();
    }
}

class Demo {
    public static void main(String[] args) {
        FileProcessor f = new FileProcessor();
        Producer p = new Producer(f);
        Consumer c = new Consumer(f);
        p.start();
        c.start();
        try {
            p.join();
            c.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("Result: " + f.getSum());
    }
}

```

Output:

```

[be2337@localhost Assignment2]$ java Demo
Result: 21
[be2337@localhost Assignment2]$

```

Number.txt

```

0 1 2 3 4 5 6
~
~
~
~

```

Q5. Consider the series $1+2+3+\dots+100$. This can be considered as $(1+3+5+\dots+99)+(2+4+6+\dots+100)$. Create two threads to compute two series in parallel (do not use simplified equation). Finally print the final sum.

Ans:

```
class Series {
    int sum = 0;
    synchronized void evenSum() {
        try {
            for(int i=2; i<=100; i=i+2) {
                sum += i;
                notify();
                if(i<100) wait();
            }
        } catch(Exception e) {
            e.printStackTrace();
        }
    }
    synchronized void oddSum() {
        try {
            for(int i=1; i<=100; i=i+2) {
                sum += i;
                notify();
                if(i<99) wait();
            }
        } catch(Exception e) {
            e.printStackTrace();
        }
    }
}

class evenSeries extends Thread {
    Series s;
    evenSeries(Series s) {
        this.s = s;
    }
}
```

```

    }

    public void run() {
        s.evenSum();
    }
}

class oddSeries extends Thread {
    Series s;

    oddSeries(Series s) {
        this.s = s;
    }

    public void run() {
        s.oddSum();
    }
}

} class Demo {
    public static void main(String[] args) {
        Series s = new Series();
        evenSeries e = new evenSeries(s);
        oddSeries o = new oddSeries(s);
        e.start();
        o.start();
        try {
            e.join();
            o.join();
        } catch (Exception E) {
            E.printStackTrace();
        }
        System.out.println("Result: " + s.sum);
    }
}

```

output:

```
[be2337@localhost Assignment2]$ javac q5.
[be2337@localhost Assignment2]$ java Demo
Result: 5050
[be2337@localhost Assignment2]$
```

Q6. Consider the following parallel binary search algorithm for series $a_1, a_2, \dots, a_n$ sorted in increasing order such that $n \bmod 10 = 0$. Element to be searched is e . a) Create $n/10$ threads $t_1, t_2, \dots, t_{n/10}$. b) Distribute the numbers among threads such that t_i will have numbers $a_i, a_{i+1}, \dots, a_{2i-1}$. c) Distribute the element e to all threads. d) Each thread searches the element e in its sub-array using binary search algorithm.

Ans:

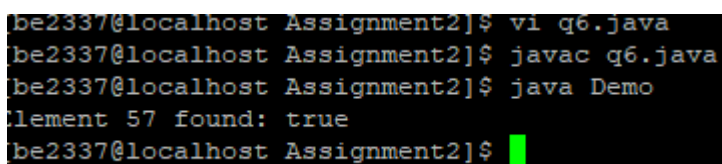
```
class ParallelBinarySearch extends Thread {
    int[] arr;
    int left, right, key;
    boolean found = false;
    ParallelBinarySearch(int[] arr, int left, int right, int key) {
        this.arr = arr;
        this.left = left;
        this.right = right;
        this.key = key;
    }
    public void run() {
        found = binarySearch(arr, left, right, key);
    }
    private boolean binarySearch(int[] arr, int left, int right, int key) {
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (arr[mid] == key) return true;
            if (arr[mid] < key) left = mid + 1;
            else right = mid - 1;
        }
        return false;
    }
}
```

```

class Demo {
    public static void main(String[] args) {
        int n = 100;
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = i + 1;
        int key = 57;
        int numThreads = n / 10;
        ParallelBinarySearch[] threads = new ParallelBinarySearch[numThreads];
        for (int i = 0; i < numThreads; i++) {
            int left = i * 10;
            int right = left + 9;
            threads[i] = new ParallelBinarySearch(arr, left, right, key);
            threads[i].start();
        }
        boolean found = false;
        for (int i=0; i<numThreads; i++) {
            try {
                threads[i].join();
                if (threads[i].found) found = true;
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        System.out.println("Element " + key + " found: " + found);
    }
}

```

Ans:



```

be2337@localhost Assignment2]$ vi q6.java
be2337@localhost Assignment2]$ javac q6.java
be2337@localhost Assignment2]$ java Demo
Element 57 found: true
be2337@localhost Assignment2]$

```

Q7. Write a Java program using threading technology and print the thread index and location where the element has been found

Ans:

```
class ParallelBinarySearch extends Thread {  
    int[] arr;  
    int left, right, key, index = -1, threadId;  
    boolean found = false;  
    ParallelBinarySearch(int[] arr, int left, int right, int key, int threadId) {  
        this.arr = arr;  
        this.left = left;  
        this.right = right;  
        this.key = key;  
        this.threadId = threadId;  
    }  
    public void run() {  
        index = binarySearch(arr, left, right, key);  
        if (index != -1) found = true;  
    }  
    private int binarySearch(int[] arr, int left, int right, int key) {  
        while (left <= right) {  
            int mid = left + (right - left) / 2;  
            if (arr[mid] == key) return mid;  
            if (arr[mid] < key) left = mid + 1;  
            else right = mid - 1;  
        }  
        return -1;  
    }  
}  
  
class Demo {  
    public static void main(String[] args) {  
        int n = 100;  
        int[] arr = new int[n];  
        for (int i = 0; i < n; i++) arr[i] = i + 1;  
        int key = 57;
```

```

int numThreads = n / 10;
ParallelBinarySearch[] threads = new ParallelBinarySearch[numThreads];
for (int i = 0; i < numThreads; i++) {
    int left = i * 10;
    int right = left + 9;
    threads[i] = new ParallelBinarySearch(arr, left, right, key, i);
    threads[i].start();
}
boolean found = false;
for (ParallelBinarySearch thread : threads) {
    try {
        thread.join();
        if (thread.found) {
            found = true;
            System.out.println("Element " + key + " found by thread " +
thread.threadId
                                + " at index " + thread.index);
        }
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
if (!found) {
    System.out.println("Element " + key + " not found.");
}
}
}

```

Ans:

```

[be2337@localhost Assignment2]$ java Demo
Element 57 found by thread 5 at index 56
[be2337@localhost Assignment2]$ █

```